Komunikasi Antarservice via Message Queue

Marketplace UMKM - kelanjutan Lab 09

Bambang Herlambang
NRP 5803024019
Proyek Marketplace UMKM (Kelompok D)

Panduan menetapkan praktikum berkelompok. Identitas di atas adalah pemilik repository; identitas anggota lain belum diberikan.

Verifikasi	Hasil aktual
Suite otomatis Lab 09 + Lab 10	75/75 lulus, 0 gagal
Integrasi RabbitMQ asli	8 skenario lulus; seluruh proses demo sudah ditutup
Broker	RabbitMQ 3.13.7; Docker, volume persisten
Waktu awal run	04-11-2026 16:39:42 UTC+7

Repository yang sama dengan Lab 09:

github.com/academicbambang/prakwebservice-lab09-microservices

Versi pengumpulan: branch codex/lab10-message-queue

Ringkasan

Order Service tetap memastikan stok melalui HTTP, lalu menerbitkan order.created ke RabbitMQ. Consumer memproses email simulasi secara terpisah. Pengujian membuktikan antrean menahan pesan ketika consumer mati, membagi beban antarkerja, menyebarkan salinan melalui fanout, serta mengirim ulang pesan yang belum di-ack.

Bukti lokal ini tidak menyatakan bahwa Lab 10 sudah dideploy ke Render. URL dan bukti deployment lama di repository adalah arsip Lab 09.

1. Arsitektur dan batas service

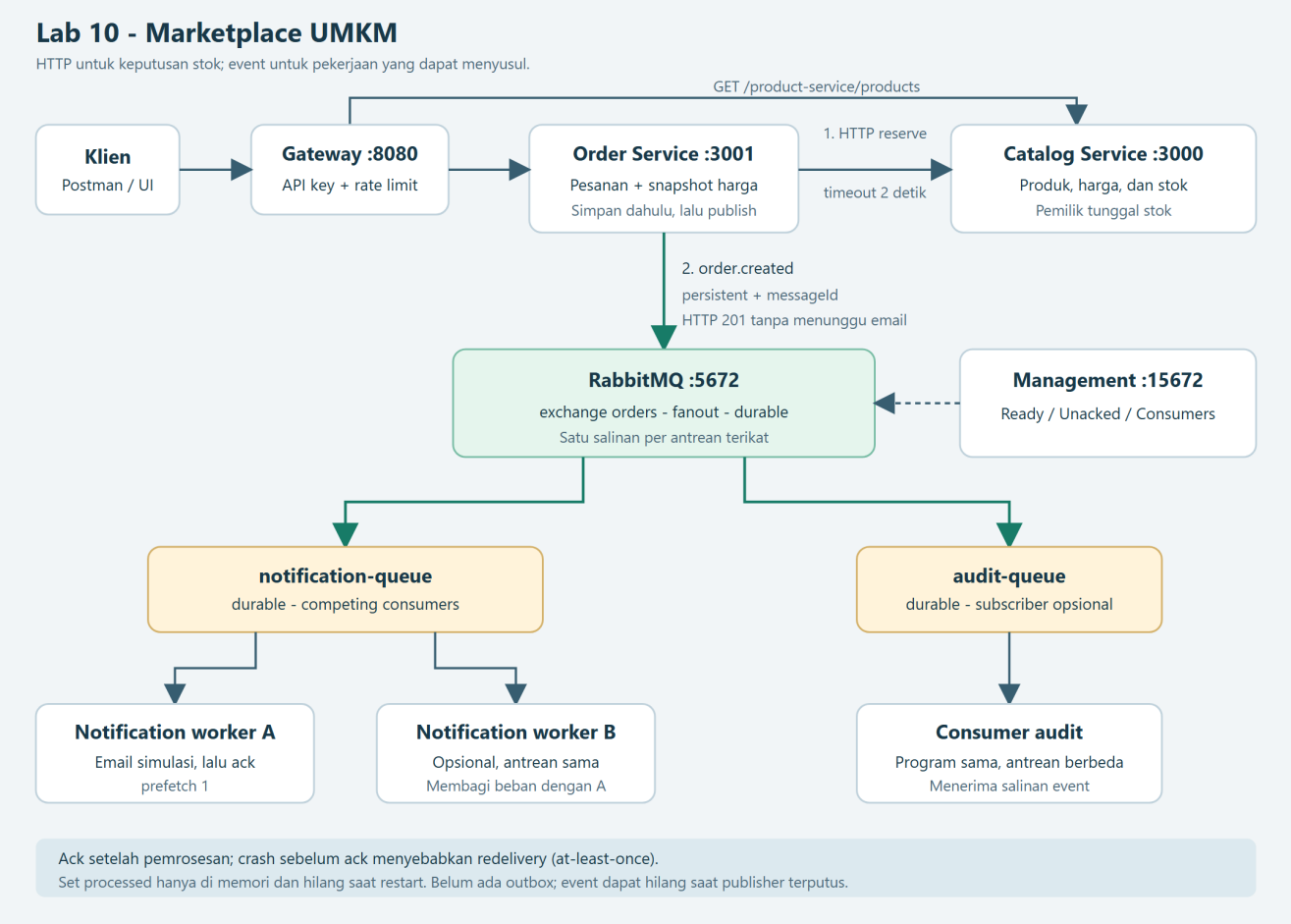


Diagram diperbarui dengan RabbitMQ, exchange orders, antrean notification/audit, dan consumer terpisah.

Komponen	Data dan tanggung jawab
Catalog Service	Produk, harga, stok di memori; hanya service ini yang mengubah stok.
Order Service	Pesanan dan snapshot harga; reserve HTTP dengan timeout 2 detik; publish sesudah order disimpan.
RabbitMQ	Exchange orders fanout dan antrean durable; pesan persistent; dashboard :15672.
Notification Service	Email simulasi; manual ack, prefetch 1; Set messageId hanya selama proses hidup.

2. Menjalankan dan kontrak event

Persiapan dan terminal

Gunakan Node.js 22+ dan Docker Desktop Linux. Dari root repository: **npm ci**, **npm run setup**, lalu **npm run broker:up**. Dashboard lokal http://127.0.0.1:15672 memakai akun demo bawaan RabbitMQ sesuai panduan; kredensial cloud hanya dalam environment AMQP_URL.

Terminal 1 menjalankan **npm run start:notification** terlebih dahulu untuk membuat queue dan binding. Terminal 2 menjalankan **npm start**; tunggu log publisher ready. Terminal 3 menjalankan **npm run test:smoke** atau memakai collection Postman Lab 10 dengan API key lokal.

Jalankan **npm test** dan **npm run test:lab10:evidence** untuk suite dan demo terisolasi. Screenshot otomatis memerlukan **npx playwright install chromium** sekali. Demo memilih port HTTP bebas dan virtual host RabbitMQ baru; tidak menghapus antrean pengguna.

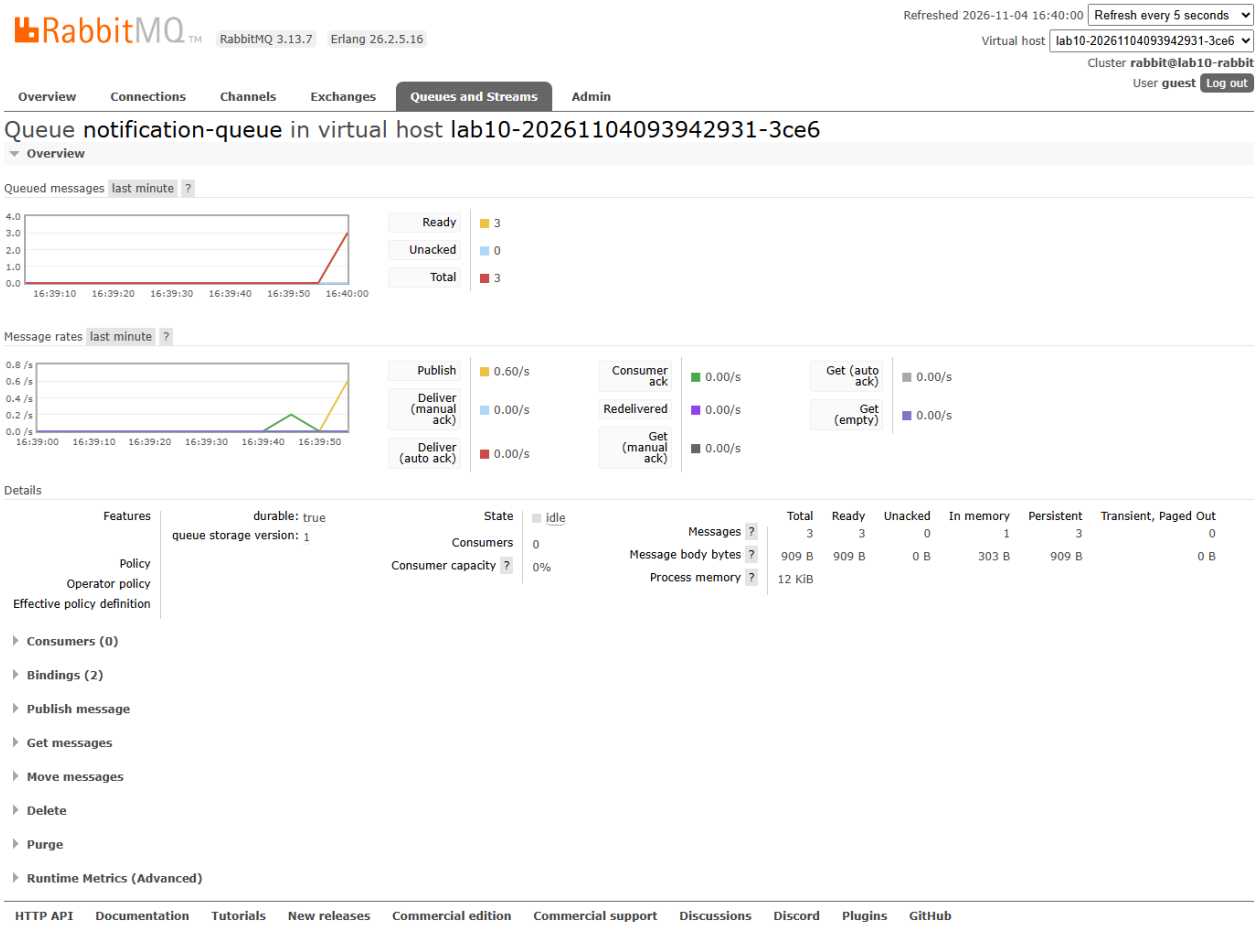
Bagian	Isi
Exchange dan event	orders / fanout / durable; type order.created
Payload data	id, customerName, items (productId, name, quantity, unitPrice, subtotal), total, status, createdAt
Properties AMQP	messageId = order-${order.id}; persistent=true; contentType=application/json
HTTP input	customerName dan items [{productId: "prod-001", quantity: 1}] sesuai API Lab 09
Ack	Setelah email simulasi; JSON/event invalid di-nack tanpa requeue

Confirm callback memeriksa penerimaan publikasi oleh broker tanpa menunggu consumer. mandatory=true mendeteksi event tanpa queue tujuan. Nilai false dari channel.publish berarti buffer penuh, bukan pesan ditolak. Reconnect publisher tiap 3 detik; tidak ada replay otomatis untuk event hilang.

Data katalog, order, dan Set processed masih in-memory. Consumer tidak terhubung ke SMTP. Detail perintah eksperimen dan konfigurasi terdapat pada docs/lab10/README.md.

3. Consumer mati: Ready = 3

Setelah queue terikat, hanya Notification Service dihentikan. Tiga order quantity 1 tetap menerima HTTP 201. Statistik broker menunjukkan Ready=3, Unacked=0, Consumers=0; publisher, order, gateway, dan katalog tetap berjalan.



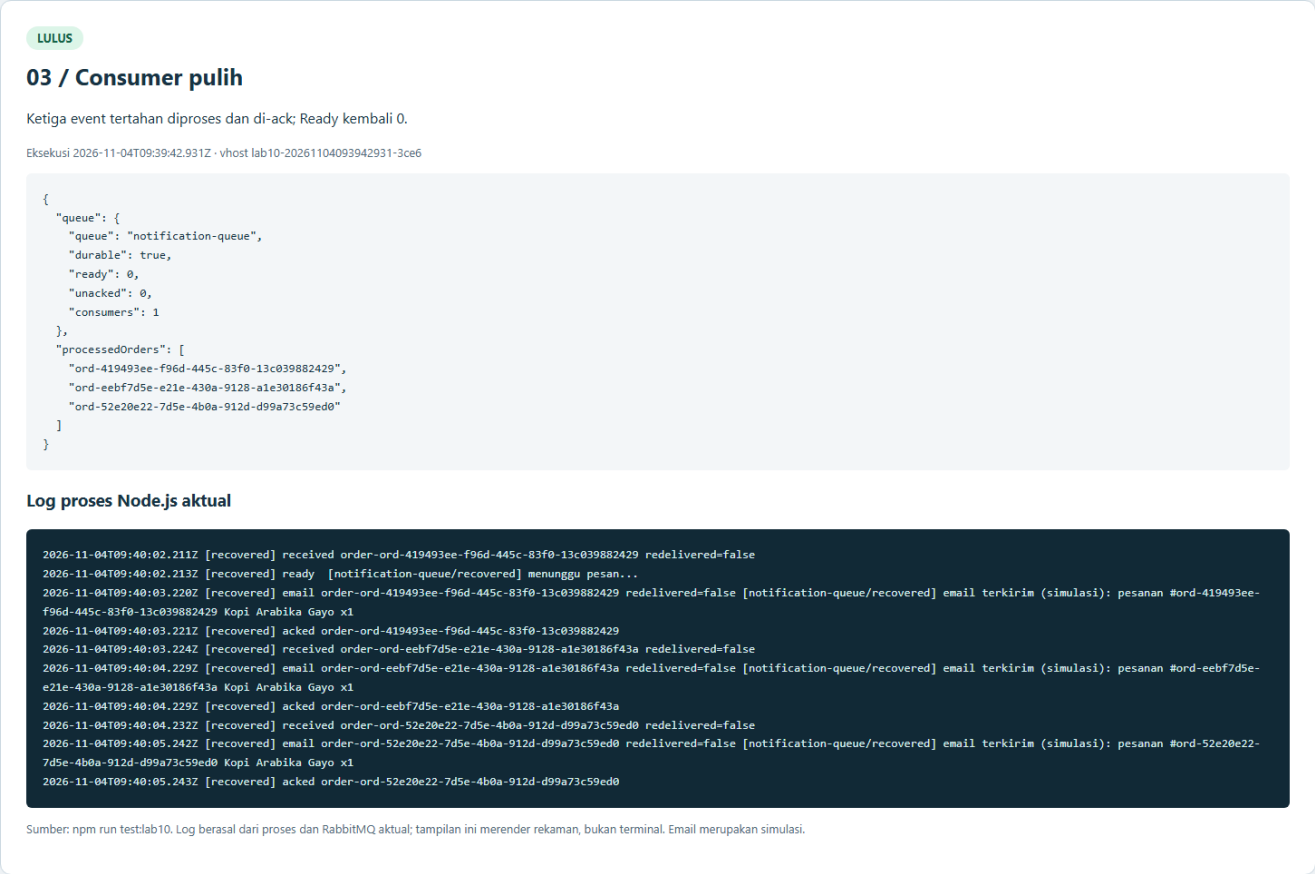
Screenshot RabbitMQ Management UI asli saat consumer mati. Virtual host terisolasi run ini ditampilkan di dashboard.

Virtual host run: lab10-20261104093942931-3ce6

Pada prosedur manual, dashboard memakai vhost /; demo otomatis memakai vhost baru agar tidak mencampur data dengan pengujian pengguna. Screenshot diambil sebelum consumer pemulihan dijalankan.

4. Consumer kembali: pesan tertahan diproses

Consumer dinyalakan kembali dan menerima tiga event yang tertahan. Ketiganya diproses dan di-ack, dengan ID yang sama seperti tiga order ketika consumer mati. Setelah selesai, Ready=0 dan Unacked=0.



LULUS

03 / Consumer pulih

Ketiga event tertahan diproses dan di-ack; Ready kembali 0.

Eksekusi 2026-11-04T09:39:42.931Z · vhost lab10-20261104093942931-3ce6


```
{
  "queue": {
    "queue": "notification-queue",
    "durable": true,
    "ready": 0,
    "unacked": 0,
    "consumers": 1
  },
  "processedOrders": [
    "ord-419493ee-f96d-445c-83f0-13c039882429",
    "ord-eebf7d5e-e21e-430a-9128-a1e30186f43a",
    "ord-52e20e22-7d5e-4b0a-912d-d99a73c59ed0"
  ]
}
```


Log proses Node.js aktual


```
2026-11-04T09:40:02.211Z [recovered] received order-ord-419493ee-f96d-445c-83f0-13c039882429 redelivered=false
2026-11-04T09:40:02.213Z [recovered] ready  [notification-queue/recovered] menunggu pesan...
2026-11-04T09:40:03.220Z [recovered] email order-ord-419493ee-f96d-445c-83f0-13c039882429 redelivered=false [notification-queue/recovered] email terkirim (simulasi): pesanan #ord-419493ee-f96d-445c-83f0-13c039882429 Kopi Arabika Gayo x1
2026-11-04T09:40:03.221Z [recovered] acked order-ord-419493ee-f96d-445c-83f0-13c039882429
2026-11-04T09:40:03.224Z [recovered] received order-ord-eebf7d5e-e21e-430a-9128-a1e30186f43a redelivered=false
2026-11-04T09:40:04.229Z [recovered] email order-ord-eebf7d5e-e21e-430a-9128-a1e30186f43a redelivered=false [notification-queue/recovered] email terkirim (simulasi): pesanan #ord-eebf7d5e-e21e-430a-9128-a1e30186f43a Kopi Arabika Gayo x1
2026-11-04T09:40:04.229Z [recovered] acked order-ord-eebf7d5e-e21e-430a-9128-a1e30186f43a
2026-11-04T09:40:04.232Z [recovered] received order-ord-52e20e22-7d5e-4b0a-912d-d99a73c59ed0 redelivered=false
2026-11-04T09:40:05.242Z [recovered] email order-ord-52e20e22-7d5e-4b0a-912d-d99a73c59ed0 redelivered=false [notification-queue/recovered] email terkirim (simulasi): pesanan #ord-52e20e22-7d5e-4b0a-912d-d99a73c59ed0 Kopi Arabika Gayo x1
2026-11-04T09:40:05.243Z [recovered] acked order-ord-52e20e22-7d5e-4b0a-912d-d99a73c59ed0
```

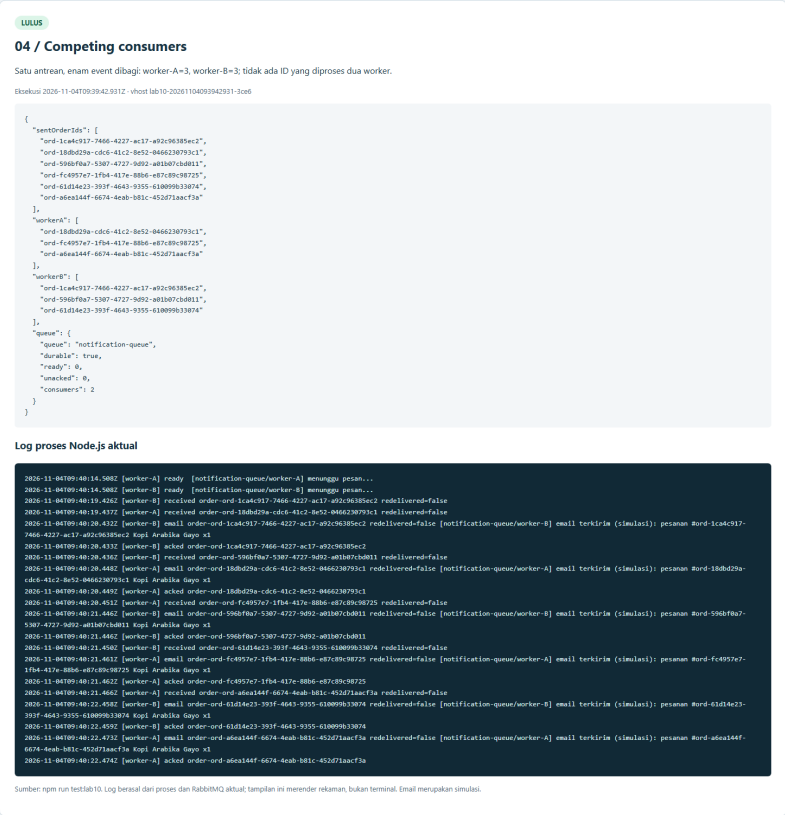

Sumber: npm run test:lab10. Log berasal dari proses dan RabbitMQ aktual; tampilan ini merender rekaman, bukan terminal. Email merupakan simulasi.

Tangkapan halaman rekaman log proses aktual; bukan screenshot terminal. Setiap email simulasi diikuti ack.

Pengirim dan penerima tidak harus aktif bersamaan jika broker aktif dan queue/binding telah dibuat. Exchange sendiri bukan tempat penyimpanan: tanpa antrean terikat, pesan fanout tidak dapat ditahan.

5. Competing consumers dan publish/subscribe

Eksperimen	Pengamatan aktual	Makna
Antrean sama	6 event dibagi 3 ke worker-A dan 3 ke worker-B; ID tidak tumpang tindih	Berbagi beban; satu salinan diproses satu worker pada satu waktu
Antrean berbeda	2 event diterima notification-queue dan audit-queue; masing-masing memproses 2	Fanout mengirim satu salinan ke setiap antrean terikat



Pembagian enam event pada dua consumer, satu antrean. Screenshot penuh dan log mentah tersedia di folder bukti.

prefetch(1) membatasi satu pesan unacked per worker. Pembagian persis 3+3 adalah hasil run ini, bukan jaminan keseimbangan untuk semua waktu pemrosesan. Pada crash, pesan dapat berpindah worker melalui redelivery.

Consumer audit memakai program simulasi yang sama dengan argumen audit-queue, bukan database audit produksi.

6. Crash sebelum ack dan email dua kali

Semua consumer notification-queue dihentikan, lalu satu consumer dimulai dengan --crash. Satu order dibuat; consumer mencetak email simulasi lalu keluar dengan kode 1 sebelum ack. Ketika consumer biasa dimulai kembali, pesan yang sama dikirim ulang.

LULUS

06 / Crash sebelum ack & redelivery

MessageId yang sama menghasilkan email simulasi dua kali; proses kedua menerima redelivered=true.

Eksekusi 2026-11-04T09:39:42.931Z · vhost lab10-20261104093942931-3ce6


```
{
  "orderId": "ord-0e1f5944-1f8c-407c-9fb7-90644dfd90fb",
  "messageId": "order-ord-0e1f5944-1f8c-407c-9fb7-90644dfd90fb",
  "crashExitCode": 1,
  "queueAfterCrash": {
    "queue": "notification-queue",
    "durable": true,
    "ready": 1,
    "unacked": 0,
    "consumers": 0
  },
  "queueAfterRecovery": {
    "queue": "notification-queue",
    "durable": true,
    "ready": 0,
    "unacked": 0,
    "consumers": 1
  },
  "simulatedEmailCount": 2,
  "explanation": "Set processed hanya di memori. Setelah restart, ID lama hilang dan pesan diproses ulang."
}
```


Log proses Node.js aktual

```
2026-11-04T09:40:49.585Z [crash-before-ack] ready  [notification-queue/crash-before-ack] menunggu pesan...
2026-11-04T09:40:49.660Z [crash-before-ack] received order-ord-0e1f5944-1f8c-407c-9fb7-90644dfd90fb redelivered=false
2026-11-04T09:40:50.666Z [crash-before-ack] email order-ord-0e1f5944-1f8c-407c-9fb7-90644dfd90fb redelivered=false [notification-queue/crash-before-ack] email terkirim (simulasi): pesanan
#ord-0e1f5944-1f8c-407c-9fb7-90644dfd90fb Kopi Arabika Gayo x1
2026-11-04T09:40:50.667Z [crash-before-ack] crash order-ord-0e1f5944-1f8c-407c-9fb7-90644dfd90fb Crash sebelum ack! Set processed akan hilang saat proses mati.
2026-11-04T09:40:59.492Z [after-crash] received order-ord-0e1f5944-1f8c-407c-9fb7-90644dfd90fb redelivered=true
2026-11-04T09:40:59.495Z [after-crash] ready  [notification-queue/after-crash] menunggu pesan...
2026-11-04T09:41:00.496Z [after-crash] email order-ord-0e1f5944-1f8c-407c-9fb7-90644dfd90fb redelivered=true [notification-queue/after-crash] email terkirim (simulasi): pesanan #ord-
0e1f5944-1f8c-407c-9fb7-90644dfd90fb Kopi Arabika Gayo x1
2026-11-04T09:41:00.496Z [after-crash] acked order-ord-0e1f5944-1f8c-407c-9fb7-90644dfd90fb
```

Sumber: npm run test:lab10. Log berasal dari proses dan RabbitMQ aktual; tampilan ini merender rekaman, bukan terminal. Email merupakan simulasi.

Dua log email memiliki messageId dan orderId sama. Pemrosesan kedua menunjukkan redelivered=true dan kemudian ack.

MessageId: order-ord-0e1f5944-1f8c-407c-9fb7-90644dfd90fb

Set processed hilang saat proses mati sehingga duplikat tidak terdeteksi setelah restart. Broker memberi redelivery untuk pesan yang belum di-ack: at-least-once, bukan jaminan email tepat sekali.

7. Verifikasi tambahan dan batas konsistensi

Pengujian	Hasil aktual
Alur normal	HTTP 201 diterima sebelum email simulasi selesai.
Duplikat dalam proses sama	MessageId lama di-ack dan dilewati; tidak ada email baru.
JSON rusak	nack(requeue=false); pesan tidak berputar tanpa batas.
Publisher terputus	Management API menutup hanya koneksi AMQP publisher; order tetap 201, event tidak terkirim.
Publisher pulih	Reconnect otomatis; event untuk order baru terkirim, event order saat putus tidak direplay.
Suite Lab 09 + Lab 10	75 test, 75 lulus, 0 gagal.

Skenario gangguan publisher tidak mematikan seluruh broker. Unit test juga memeriksa kegagalan koneksi awal, channel error, backpressure, ack/nack, dan struktur event. Test Lab 09 yang tetap lulus mencakup stok, timeout, isolasi kegagalan, API key, gateway, dan frontend.

Durable queue dan persistent message melindungi pesan yang sudah diantrekan dengan konfigurasi yang sesuai; keduanya tidak membuat penyimpanan order dan publish menjadi satu transaksi. Kegagalan sebelum publish dapat meninggalkan order tanpa notifikasi. Publisher confirms tidak berarti email sudah terkirim dan tidak menutup celah ini.

Untuk produksi: simpan order dan transactional outbox dalam satu transaksi database, relay dengan retry dan confirms, lalu gunakan consumer idempotent. Untuk side effect database, unique (consumer_name, message_id) dan perubahan bisnis harus berada dalam transaksi yang sama, commit sebelum ack. Email eksternal membutuhkan outbox pengiriman dan idempotency key penyedia; tabel processed saja tidak menjamin exactly-once.

Perbaikan produksi tersebut adalah rancangan lanjutan, belum fitur implementasi praktikum ini. Consumer sengaja memakai Set di memori untuk memperlihatkan eksperimen crash sesuai panduan.

8. Penerapan proyek dan refleksi

Aspek	Marketplace UMKM
Interaksi	Pemberitahuan pesanan baru setelah checkout.
Event	order.created; ID order, snapshot items/harga, quantity, total, status, waktu.
Producer	Order Service, setelah stok direservasi dan order disimpan.
Consumer	Notification Service; subscriber audit opsional pada antrean berbeda.
Alasan asinkron	Email bisa menyusul; consumer email mati tidak perlu membuat checkout gagal. Stok tetap sinkron karena hasilnya dibutuhkan untuk menerima order.

Refleksi (5 kalimat)

Jika RabbitMQ mati saat order dibuat, publisher mencatat bahwa event tidak terkirim, sementara order tetap tersimpan dan klien tetap menerima HTTP 201. Hal itu dapat membuat data pesanan dan notifikasi tidak konsisten, sehingga pengembangan selanjutnya memerlukan penyimpanan order bersama outbox dalam satu transaksi serta relay yang mencoba ulang. Consumer yang idempotent harus memakai messageId unik di penyimpanan tahan restart dan mencatat ID beserta side effect database secara atomik sebelum ack, bukan hanya menggunakan Set dalam memori. Untuk email eksternal, diperlukan outbox pengiriman dan dukungan idempotency key dari penyedia karena transaksi database sendiri tidak menjamin email terkirim tepat sekali. Panggilan sinkron tetap lebih tepat untuk reservasi stok dan pengambilan harga saat checkout karena Order Service harus mengetahui hasilnya sebelum menerima pesanan, sedangkan notifikasi dapat diproses kemudian.

Sumber dan pengumpulan

Laporan lengkap: docs/lab10/README.md. Bukti aktual: docs/evidence/lab10/results.json, execution.txt, tests.txt, screenshot, dan log tiap proses. Collection manual: docs/postman/lab10.postman_collection.json. Pengumpulan mengikuti panduan sebelum Lab 11.

RabbitMQ - Work Queues

RabbitMQ - Publish/Subscribe

amqplib - Channel API

Panduan Praktikum Web Service Lab 10 - Philipus Suryo Subandoro, bahan tugas yang diberikan.